ДЕПАРТАМЕНТ ОБРАЗОВАНИЯ И НАУКИ ГОРОДА МОСКВЫ
ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ГОРОДА МОСКВЫ
«КОЛЛЕДЖ СВЯЗИ №54»
ИМЕНИ П.М. ВОСТРУХИНА

ПРАКТИЧЕСКАЯ РАБОТА № 3

МДК.05.03

Тестирование информационных систем

Тема: «Bash, часть 1: отдельные тесты и общий запуск»

Выполнила

ст. Гр. 4-ИСП9-46

Миронова Анна Андреевна

Москва, 2026

СОДЕРЖАНИЕ

Цель работы 3

Выполненная работа 3

Средства разработки 3

Анализ сложности 3

Результаты выполнения 4

Ответы на контрольные вопросы 5

Проверка результатов 6

Заключение 7

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ И ИНТЕРНЕТ-ИСТОЧНИКОВ 8

Листинги программ 9

Цель работы

Проверить корректность работы ИС и научиться обнаруживать воспроизводимые дефекты.

Условие: practice_rose_test_bash_system_part_0.md.

Выполненная работа

Созданы отдельные test_file.sh, test_lines.sh, test_content.sh, test_rights.sh, test_process.sh, test_port.sh, test_http.sh, test_http_code.sh, test_http_content.sh, test_no_errors.sh. Дополнительно: каталог логов, формат CSV каждой строки, уникальные ID, запись в каталог логов и время ответа. check.sh сохраняет дату и результат каждой проверки. run_all.sh не теряет код ошибки дочернего скрипта.

Средства разработки

Python 3.14.6, Flask 3.1.2, SQLite 3.53.3. Проверки выполнены в Arch Linux/WSL. Запуск в POCA и Python 3.8 отдельно не проверялся.

Порядок запуска: инструкция.

Анализ сложности

Файловые проверки test имеют O(1) метаданных. Чтение CSV O(B), поиск дубликатов ID с sort O(n log n). HTTP-проверки ограничены max-time/timeout, чтобы недоступный сервер не зависал на неопределённый срок.

Дубликат задания и самостоятельная работа

Файлы practice_rose_test_bash_system.md и practice_rose_test_bash_system_part_0.md побайтно совпадают; им соответствует одно решение. Дополнительные задания выполнены: logs существует и доступен для записи; число строк равно 3, включая заголовок; каждая строка содержит 3 поля; ID уникальны. Условие отсутствия error проверяется только после успешного получения HTTP-тела. Это исправляет ложный PASS из отрицания grep в исходном примере.

Результаты выполнения

Результат проверки показан на рисунке 1.

```
$ bash testing/bash_basics/run_all.sh
$ tail -n 23
TOTAL=1 PASS=1 FAIL=0
2026-11-11 07:25:02 PASS: HTTP-код 200
TOTAL=1 PASS=1 FAIL=0
2026-11-11 07:25:02 PASS: HTTP содержит Ivan
TOTAL=1 PASS=1 FAIL=0
2026-11-11 07:25:02 PASS: В файле ровно 3 строки
TOTAL=1 PASS=1 FAIL=0
2026-11-11 07:25:02 PASS: Каталог логов доступен на запись
TOTAL=1 PASS=1 FAIL=0
2026-11-11 07:25:02 PASS: Каталог логов существует
TOTAL=1 PASS=1 FAIL=0
2026-11-11 07:25:02 PASS: Успешный ответ без error
TOTAL=1 PASS=1 FAIL=0
2026-11-11 07:25:02 PASS: TCP-порт открыт
TOTAL=1 PASS=1 FAIL=0
2026-11-11 07:25:02 PASS: Собственный процесс сервера запущен
TOTAL=1 PASS=1 FAIL=0
2026-11-11 07:25:02 PASS: Ответ быстрее 1 секунды
TOTAL=1 PASS=1 FAIL=0
2026-11-11 07:25:02 PASS: Файл читается
TOTAL=1 PASS=1 FAIL=0
2026-11-11 07:25:02 PASS: ID уникальны
TOTAL=1 PASS=1 FAIL=0

Код завершения: 0
```

Рисунок 1 – Результат отдельных Bash-проверок при работающем сервере

Результат проверки показан на рисунке 2.

```
$ bash testing/bash_basics/run_all.sh
$ tail -n 23
TOTAL=1 PASS=0 FAIL=1
2026-11-11 07:25:05 FAIL: HTTP-код 200
TOTAL=1 PASS=0 FAIL=1
2026-11-11 07:25:05 FAIL: HTTP содержит Ivan
TOTAL=1 PASS=0 FAIL=1
2026-11-11 07:25:05 PASS: В файле ровно 3 строки
TOTAL=1 PASS=1 FAIL=0
2026-11-11 07:25:05 PASS: Каталог логов доступен на запись
TOTAL=1 PASS=1 FAIL=0
2026-11-11 07:25:05 PASS: Каталог логов существует
TOTAL=1 PASS=1 FAIL=0
2026-11-11 07:25:05 FAIL: Успешный ответ без error
TOTAL=1 PASS=0 FAIL=1
2026-11-11 07:25:05 FAIL: TCP-порт открыт
TOTAL=1 PASS=0 FAIL=1
2026-11-11 07:25:05 FAIL: Собственный процесс сервера запущен
TOTAL=1 PASS=0 FAIL=1
2026-11-11 07:25:05 FAIL: Ответ быстрее 1 секунды
TOTAL=1 PASS=0 FAIL=1
2026-11-11 07:25:05 PASS: Файл читается
TOTAL=1 PASS=1 FAIL=0
2026-11-11 07:25:06 PASS: ID уникальны
TOTAL=1 PASS=1 FAIL=0

Код завершения: 1
```

Рисунок 2 – Обнаружение недоступности сервера Bash-тестами

Ответы на контрольные вопросы

1. Что такое код возврата и как его проверить?

Число, возвращаемое процессом оболочке: 0 — успех, ненулевое значение — ошибка. Сразу после команды выполните echo “$?”. В наших агрегаторах наличие хотя бы одного FAIL даёт exit 1.

2. Зачем shift в check?

Первый аргумент — название проверки. shift удаляет его из позиционных параметров; “$@” становится выполняемой командой с её аргументами.

3. Почему >/dev/null 2>&1 скрывает весь вывод?

Сначала stdout перенаправляется в /dev/null, затем stderr связывается с уже перенаправленным stdout. В реализации отчёта диагностика сохраняется в служебных файлах вместо потери.

4. Файл существует и сервер отдаёт файл — одно и то же?

Нет. Файл может быть на диске при остановленном сервере, неправильном URL, запрете доступа или неверном корне публикации. Проверки диска и HTTP независимы.

5. Для чего timeout?

Ограничивает ожидание соединения. Без него недоступный адрес может задержать весь прогон на системный TCP-таймаут.

6. Что делает ! перед командой?

Инвертирует код успеха: 0 становится 1, ненулевой код становится 0. Нельзя отрицать поиск error в пустом ответе, не проверив успешность curl: иначе недоступный сервер ошибочно даёт PASS.

7. Почему после остановки сервера проходят некоторые проверки?

Проверки локального файла, его структуры и прав не зависят от HTTP-процесса. Сетевые проверки должны провалиться; это подтверждено отдельным прогоном.

Проверка результатов

Результат проверки показан на рисунке 3.



Рисунок 3 – Журнал фактических запусков cron

Заключение

Каждая базовая проверка выделена в отдельный Bash-скрипт: существование и содержимое файла, права доступа, процесс, порт и HTTP-ответ. Дополнительные проверки контролируют структуру CSV, уникальность ID и возможность записи в каталог журналов. Агрегатор запускает скрипты последовательно и учитывает их коды завершения.

Набор проверен при работающем и остановленном локальном сервере. Во втором случае сетевые тесты завершились ошибкой, а проверки файлов остались независимыми от сети. Отсутствие слова error оценивается только после успешной загрузки ответа: ошибка curl не должна давать ложный положительный результат. Полученные скрипты можно повторно запускать после изменения данных или состояния сервиса и сравнивать результаты по журналу.

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ И ИНТЕРНЕТ-ИСТОЧНИКОВ

1. Емелина, Е.И. Поддержка и тестирование программных модулей [Текст]: учебное пособие / Е.И. Емелина. – Москва: КНОРУС, 2025. – 267 с.
2. Жуков, Р.А. Язык программирования Python: практикум [Текст]: учебное пособие / Р.А. Жуков. – Москва: ИНФРА-М, 2026. – 216 с.
3. Старолетов, С.М. Основы тестирования и верификации программного обеспечения [Текст]: учебное пособие / С.М. Старолетов. – 3-е изд., стер. – Санкт-Петербург: Лань, 2023. – 344 с.
4. Фёдорова, Г.Н. Разработка, внедрение и адаптация программного обеспечения отраслевой направленности [Текст]: учебное пособие / Г.Н. Фёдорова. – Москва: КУРС: ИНФРА-М, 2024. – 336 с.
5. Чернышев, С.А. Основы программирования на Python [Текст]: учебное пособие для вузов / С.А. Чернышев. – 2-е изд., перераб. и доп. – Москва: Юрайт, 2024. – 349 с.

Электронные ресурсы

6. Документация платформы тестирования API Postman и утилиты Newman [Электронный ресурс]: https://learning.postman.com/docs/ (дата обращения 10.11.2026).
7. Международный комитет по квалификации тестирования программного обеспечения (ISTQB) [Электронный ресурс]: https://www.istqb.org/ (дата обращения 10.11.2026).
8. Руководство по командной оболочке GNU Bash и системным утилитам Linux [Электронный ресурс]: https://www.gnu.org/software/bash/manual/ (дата обращения 10.11.2026).

Листинги программ

Testing/bash_basics/check.sh

```
#!/usr/bin/env bash
set -u
ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
DATA="${QA_DATA:-$ROOT/data}"
RUN="${QA_RUN:-$ROOT/.logs}"
mkdir -p "$RUN"
LOG_FILE="$RUN/test_$(date +%Y%m%d_%H%M%S)_$$.log"
PASS=0
FAIL=0
check() {
    local name="$1"
    shift
    if "$@" >"$RUN/last_stdout.txt" 2>"$RUN/last_stderr.txt"; then
        echo "$(date '+%F %T') PASS: $name" | tee -a "$LOG_FILE"
        PASS=$((PASS + 1))
    else
        echo "$(date '+%F %T') FAIL: $name" | tee -a "$LOG_FILE" "$RUN/alerts.log"
        cat "$RUN/last_stderr.txt" >> "$LOG_FILE"
        FAIL=$((FAIL + 1))
    fi
}
finish() {
    echo "TOTAL=$((PASS + FAIL)) PASS=$PASS FAIL=$FAIL" | tee -a "$LOG_FILE"
    test "$FAIL" -eq 0
}
```

Testing/bash_basics/data/users.csv

```
id,name,status
1,Ivan,active
2,Maria,inactive
```

Testing/bash_basics/functions.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/check.sh"
URL="${QA_URL:-http://127.0.0.1:8080/users.csv}"
HOST="${QA_HOST:-127.0.0.1}"
PORT="${QA_PORT:-8080}"
fetch() { curl -fsS --max-time 3 "$URL"; }
file_rows() { test "$(wc -l < "$DATA/users.csv")" -eq 3; }
file_fields() { awk -F, 'NF!=3{bad=1} END{exit bad}' "$DATA/users.csv"; }
unique_ids() { test -z "$(tail -n +2 "$DATA/users.csv" | cut -d, -f1 | sort | uniq -
d)"; }
http_code() { test "$(curl -s -o /dev/null -w '%{http_code}' --max-time 3 "$URL")" =
200; }
http_content() { local body; body="$(fetch)" || return 1; [[ "$body" == *Ivan* ]]; }
no_errors() { local body; body="$(fetch)" || return 1; ! printf '%s' "$body" | grep -qi
error; }
process_alive() { test -n "${QA_PID:-}" && kill -0 "$QA_PID" 2>/dev/null; }
port_open() { timeout 2 bash -c 'exec 3<>/dev/tcp/$1/$2' bash "$HOST" "$PORT"; }
```

```
response_fast() { local seconds; seconds="$(curl -fsS -o /dev/null -w '%{time_total}'
--max-time 3 "$URL")" || return 1; awk -v t="$seconds" 'BEGIN{exit !(t<1)}'; }
```

Testing/bash_basics/run_all.sh

```
#!/usr/bin/env bash
ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
result=0
for script in "$ROOT"/test_*.sh; do
    bash "$script" || result=1
done
exit "$result"
```

Testing/bash_basics/run_tests.sh

```
#!/usr/bin/env bash
exec bash "$(dirname "${BASH_SOURCE[0]}")/run_all.sh" "$@"
```

Testing/bash_basics/test_content.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "Заголовок CSV" grep -q "^id,name,status$" "$DATA/users.csv"
finish
```

Testing/bash_basics/test_csv_format.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "Каждая строка содержит 3 поля" file_fields
finish
```

Testing/bash_basics/test_file.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "Файл существует и не пуст" test -s "$DATA/users.csv"
finish
```

Testing/bash_basics/test_http.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "HTTP доступен" fetch
finish
```

Testing/bash_basics/test_http_code.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "HTTP-код 200" http_code
finish
```

Testing/bash_basics/test_http_content.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "HTTP содержит Ivan" http_content
finish
```

Testing/bash_basics/test_lines.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "В файле ровно 3 строки" file_rows
finish
```

Testing/bash_basics/test_log_writable.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "Каталог логов доступен на запись" test -w "$RUN"
finish
```

Testing/bash_basics/test_logs.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "Каталог логов существует" test -d "$RUN"
finish
```

Testing/bash_basics/test_no_errors.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "Успешный ответ без error" no_errors
finish
```

Testing/bash_basics/test_port.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "TCP-порт открыт" port_open
finish
```

Testing/bash_basics/test_process.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "Собственный процесс сервера запущен" process_alive
finish
```

Testing/bash_basics/test_response_time.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
```

```
check "Ответ быстрее 1 секунды" response_fast
finish
```

Testing/bash_basics/test_rights.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "Файл читается" test -r "$DATA/users.csv"
finish
```

Testing/bash_basics/test_unique_ids.sh

```
#!/usr/bin/env bash
source "$(dirname "${BASH_SOURCE[0]}")/functions.sh"
check "ID уникальны" unique_ids
finish
```